

BIT 2106: OPERATING SYSTEMS

COURSE OUTLINE

Objectives

- Enable the student understand the operating systems in the computer system.
- Enable student identify various functions and roles of the operating systems in the computer system.

No.	Topic	Sub-topic
1.	Introduction	• Overview and Categories of Operating Systems • Functions of Operating Systems
2.	Process Management	• Process / Program Comparison • Process States, Process Control Blocks (PCB) • Symmetric Multi - processing • CPU Scheduling
3.	Process Synchronization	• Coordination and Synchronization • Critical Sections, Race Conditions, Semaphores • Mutual Exclusion and Concurrency • Continuous Assessment Test (CAT)
4.	Memory Management,	• Partitioning, Paging and Segmentation • Fetch and Replacement Policy • Virtual Memory • Memory Management Algorithms
5.	Input/Output Management	• <u>Input/Output Devices</u> • <u>Input/Output Function and Organization</u> • <u>Buffering and Disk Scheduling</u> • Assignment 1
6.	File Management	• File Organization • File and Directories • File System • Assessments (CAT & Assignment)
7.	Deadlocks	• <u>Deadlock Handling</u> • <u>Mutual exclusions conditions</u> • Assignment 2
8.	Revision	Revision and Assessments

Assessment:

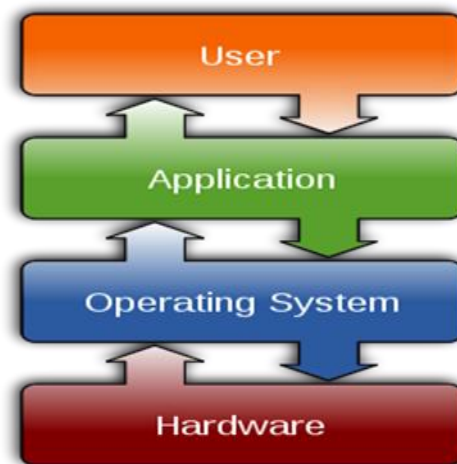
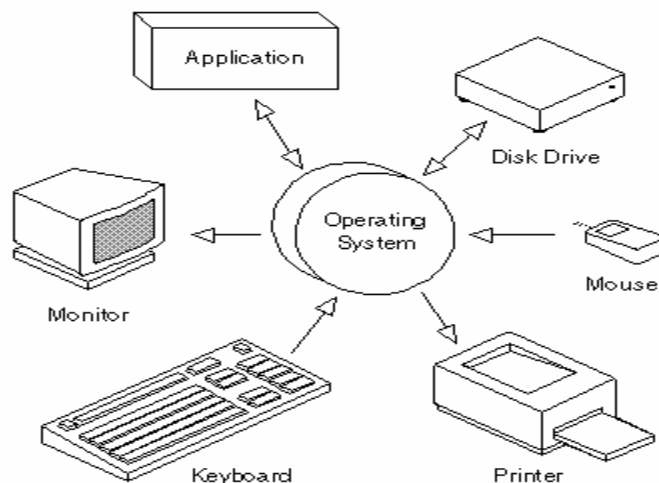
• Continuous Assessment Tests	20%
• Assignments	10%
• <u>End of Semester Exam</u>	<u>70%</u>
Total	100%

References:

- 1 Thomas Anderson (2015); *Operating Systems: Principles and Practice* (5th Edition); New York, USA
2. Greg Tomsho (2016); *Guide to Operating Systems*. (2nd Ed); McGraw Hill
3. M. Dhamdhare (2015); *Operating Systems: A Concept-based Approach*. (2nd Ed); McGraw Hill
4. The Internet

1.0 Operating System: - Introduction

1. An operating system (OS) is a collection of software that manages computer hardware resources and provides common services for computer programs. The operating system is a vital component of the system software in a computer system. An operating system is “the software that controls the hardware”. An Operating System (OS) is an interface between a computer user and hardware.
2. An operating system (OS) is software that controls a computer. It manages hardware, runs applications, provides an interface for users, and stores, retrieves, and manipulates files. In general, you can think of an operating system as the middleman between applications and hardware, between the user and hardware, and between the user and applications
3. An operating system is the low-level software that enables a user and higher-level application software to interact with a computer’s hardware and the data and other programs stored on the computer. An OS performs basic tasks, such as recognizing input from the keyboard, sending output to the display screen, keeping track of files and directories on the disk, and controlling peripheral devices such as printer
4. An operating system is the software that controls the hardware by making the hardware useable by providing an interface (or translator) between the application software (e.g. word processor) and the hardware (i.e. tells the hardware what to do and how to communicate between the parts). After being initially loaded into the computer by a boot program, it manages all the other programs in the computer.



In Summary

- ✓ Operating system is a system software that acts as an intermediary between the user of a computer and computer hardware.
- ✓ It is considered as the brain of the computer.
- ✓ It controls the internal activities of the computer hardware and provides the user interface.
- ✓ This interface enables a user to utilize the hardware resources efficiently.
- ✓ The application programs do not access the hardware resources directly
- ✓ An operating system is the first program that is loaded into the computer’s main memory, when a computer is switched on, through the process called “booting”.
- ✓ The main objective of an OS is “convenience” and “efficiency”.

Categories of Operating Systems

All operating System consists of similar component and can perform all most similar function but the method and procedure for performing these functions are different.

OPERATING SYSTEM are classified into different categories according to their different features. The following section will discuss the classification of operating system.

Single user OPERATING SYSTEM:

- In a single user operating system a single user can access the computer at a particular time.
- This system provides all the resources to the user at all the time.
- The single user operating System is divided into the following types.
 - Single user, single tasking operating System
 - Single user , multitasking operating System

Single user, single tasking operating System:

- In a single user, single tasking operating system, There is a single user to execute a program at a particular system.
- Example – MS-DOS

Single user, multitasking operating System

- In single user, multitasking OPERATING SYSTEM a single user can execute multiple programs.
- Example – A user can program different programs such as making calculations in excel sheet, printing a word document & downloading into the file from internet at the same time.

Advantage:

- The CPU has to handle only one application program at a time so that process management is easy in this environment.

Disadvantage:

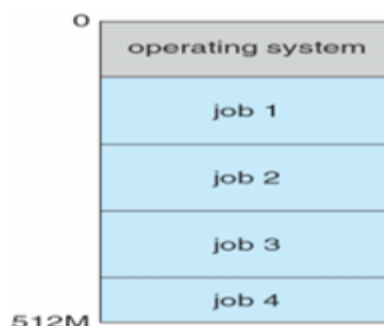
- As the operating system is handling one application at a time most of the CPU time is wasted.

Multi user OPERATING SYSTEM:

- In a multi-user operating system , multiple number of users can access different resources of a computer at a time.
- This system provides access with the help of a network. Network generally consists of various personal computers that can send and receive information to multi user mainframe computer system.
- Hence, the mainframe computer acts as the server and other personal computer act as the client for that server.

Multiprogramming OPERATING SYSTEM:

- In a multiprogramming operating System several user can execute multiple jobs by using a single CPU at the same time.
- The operating System keeps several program or job in the main memory.
- When a job is submitted to the system in a magnetic disk or job pool.
- Then some of the jobs are transferred to the main memory according to the size of the main memory.
- The CPU execute only one job which is selected by the operating System.
- When the job requires any I/O operation, then CPU



- When the job requires any I/O operation, then CPU switches to next job in the main memory i.e CPU do not have to wait for the completion of I/O operation of that job.
- When the I/O operation of that job is completed then the CPU switches to the next job after the execution of the current job.

Advantage:

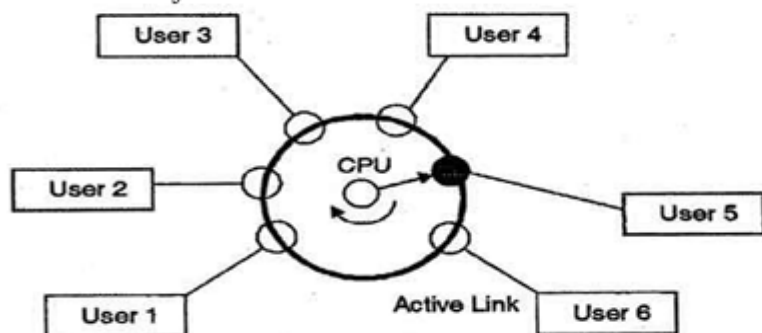
CPU utilization is more i.e the most of the time the CPU is busy.

Disadvantage:

The user can't directly interact with the system.

Time sharing Operating System:

- This is the Logical extension of multiprogramming system.
 - The CPU is multiplexed among several jobs that are kept in memory and on disk (the CPU is allocated to a job only if the job is in memory).
- ☐ Here the CPU can execute more than one job simultaneously by switching among themselves.
 - ☐ The switching process is very fast so that the user can directly interact with the system during the execution of the program.
 - ☐ This system stores multiple jobs in the main memory and CPU execute all the jobs in a sequence.
 - ☐ Generally CPU time is divided into no. of small interval known as **time slice period**.
 - ☐ Every process has to execute for the time slice period; then the CPU switch over to next process.
 - ☐ The switching process is very fast, so it seems that several processes are executed simultaneously.



In above figure the user 5 is active but user 1, user 2, user 3, and user 4 are in waiting state whereas user 6 is in ready status.

As soon as the time slice of user 5 is completed, the control moves on to the next ready user i.e. user 6. In this state user 2, user 3, user 4, and user 5 are in waiting state and user 1 is in ready state. The process continues in the same way and so on.

Advantage:

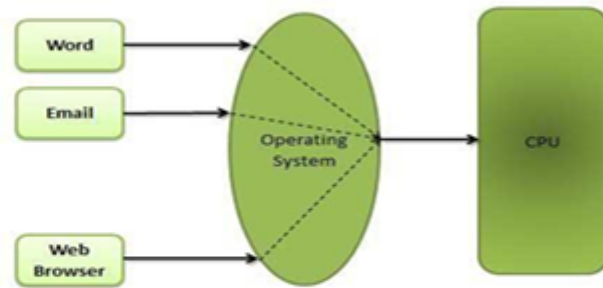
- ☐ CPU utilization is more i.e the most of the time the CPU is busy.

Disadvantage:

- ☐ The operating system is more complex due to memory management, Disk management etc.

Multitasking Operating System:

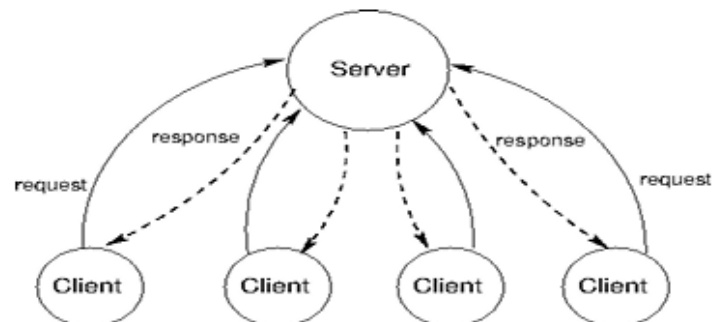
- A multi-tasking operating system allows more than one program to be running at the same time.
- E.g.-one user can open the word document and can simultaneously access the internet.
- While the processor handles only one application at a particular time it is capable of switching between the applications effectively to apparently simultaneously execute each application.
- This type of operating system is seen everywhere today and is the most common type of operating system, the Windows operating system would be an example.



Distributed Operating Systems:

- In distributed operating system, the users access remote resources in the same way as the local resources are accessed.
- Distribute the computation among several physical processors.
- Loosely coupled system – each processor has its own local memory; processors communicate with one another through various communications lines, such as high-speed buses or telephone lines.
- These systems provide features such as data and process migration.
- This operating system based on two models.
 - Client-server model
 - Peer-to-peer model

Client-server model: In this model, the client sends a request for a resource to the server and the server, in turn provides the requested resource as a response back to the client.



Peer-to-peer model: In a peer-to-peer model, all the computers behave as peers as well as clients. These peers communicate with each other for exchange of their resources.



Advantages:

- It facilitates the sharing of hardware and software resources between different processors.
- It increases reliability as failure of one node does not affect the entire network.
- It increases the computational speed of computer by sharing the workload into different nodes.
- It enable different users to communicate with each other using email.

Real time Operating System:

- In a real-time operating system a job is to be completed within the right time constraint otherwise job loses its meaning.
- These system compete a particular job in the fixed time slot in order to respond to an event quickly.
- Real time introduces for correct operation and it required to produce result within a non negotiable time period.
- Real-time systems are usually used to control complex systems that require a lot of processing like machinery and industrial systems.

- This is of 2 types:
 - Hard real time operating system
 - Soft real time operating system

Hard real-time system:

- This system completes the critical tasks within the definite interval of time constraint.
- If the critical task is not completed within the time constraint, then the system fails.
- This system has to complete all the processes within the definite deadline and a single miss leads to critical failure.
- E.g.: -Pace maker, flight control system(any miss in deadline leads to crash).

Soft real-time system:

- These systems are not affected by the lapse of time interval and do not cause any critical failure.
- E.g.: -Live video streaming.

Embedded Operating System

An embedded operating system usually has the operating system built into the computer (component), and is used to control external hardware. There is little or no application software in an embedded system. Examples here are the PDAs that everybody seems to have, and of course the computers built into DVDs, microwaves, smart televisions, medical equipment, most cars, etc. Windows CE is an example of an embedded operating system.

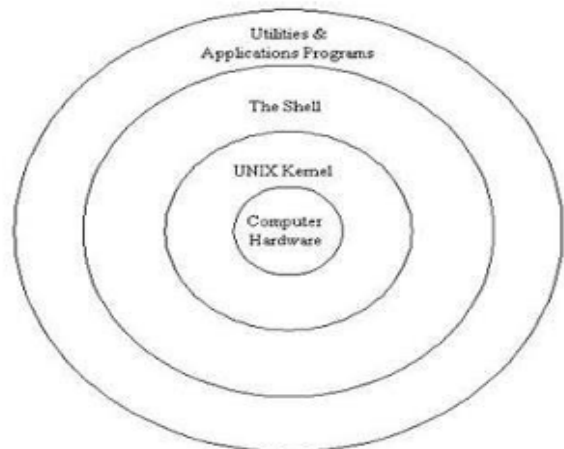
OPERATING SYSTEMS STRUCTURE

The structure of Operating System comprises of 4 layers.

- Hardware
- Kernel
- System call interface(shell)
- Application programs

Kernel:

- It is the vital part of the operating system. It interacts directly with the hardware of a computer.
- Programs interact with the kernel through 10 system calls.
- **System call:-** The *system call* provides an interface to the operating system services.
- System call tells the kernel to carry out various asks for the program such as opening a file, writing to a file, obtaining information about a file, executing a program, terminating a process etc.



- The main function of the **kernel** are
 - ❖ To manage computer memory
 - ❖ To maintain file system
 - ❖ Allocation of resources
 - ❖ Control access to the computer
 - ❖ Handle interrupts

System Call Interface(Shell):

- Shell is command line interpreter which interprets the commands given by the user.
- It is software or program which acts as a mediator between kernel and user.
- The shell read the commands, what you typed at command line and interprets them and sends request to execute a program. That's why shell is called as command line interpreter.

Utility and application programs:

- Utility programs help **manage, maintain and control computer resources**. These programs are available to help you with the day-to-day chores associated with personal computing and to keep your system running at **peak performance**.
- Application software is all the computer software that causes a computer to perform useful tasks beyond the running of the computer itself.
- Examples of application programs include word processors; database programs; Web browsers; development tools; drawing, paint, and image editing programs; and communication programs.

Hardware:

- Computer hardware refers to the physical parts or components of a computer such as the monitor, mouse, keyboard, computer data storage, hard drive disk (HDD), system unit (graphic cards, sound cards, memory, motherboard and chips), etc. all of which are physical objects that can be touched

2.0 FUNCTIONS OF OPERATING SYSTEM.

An operating system has a complex mixture of diversified functions. Operating systems is the resource manager. At the simplest level, an operating system does two things:

- One, it manages the hardware and software resources of the system. In a desktop computer, these resources include such things as the processor, memory, disk space and more (On a cell phone, they include the keypad, the screen, the address book, the phone dialer, the battery and the network connection).
- It provides a stable, consistent way for applications to deal with the hardware without having to know all the details of the hardware

The main resource is computer hardware in the form of processors, storage, input/output devices, communication devices, and data. Some of the functions are: implementing the user interface, sharing hardware among users, allowing users to share data among themselves, preventing users from interfering

with one another, scheduling resources among users, facilitating input/output, recovering from errors, accounting for resource usage, facilitating parallel operations, organizing data for secure and rapid access, and handling network communications.

The operating systems generally accomplish these goals by running processes in low privilege and providing service calls that invoke the operating system kernel in high-privilege state.

1) It acts as an Command Interpreter:

- Generally the CPU cannot understand the commands given by user. It is the function of operating System to translate this command (human understandable) into m/c understandable instructions that the system (CPU) can understand.
- After the execution of instructions by CPU, it retranslates the o/p back into a human understandable language.
- To execute the user jobs, the Operating System interacts with the computer hardware.

2) It acts as the Resource Manager:

- An Operating System acts as a resource manager in two ways
 - Time multiplexing
 - Space multiplexing
- In time multiplexing the different resources (hardware or software) can be shared among different users for a optimal or fixed time slot.
- e.g. the Operating System allocates a resources such as CPU to program A for a fixed time slot. When the time slot of process A is over, the CPU is allocated to another program B. If program A needs more CPU attention, then the CPU again allocated to program A after the time slice period allocated to program B is over.
- In space multiplexing, different resources are shared at the same time among different programs .e.g. sharing of hard disk and main memory by different users at the same time.

3) Memory Management:

- It keeps track of the resources (memory), what part of memory is in use and by whom, which part of the memory is not in use.
- Decides which processes are to be loaded when memory space is available.
- Allocation and de allocation of memory

4) Process Management:

- A process(task) is an instance of a program in execution. A program is just a passive entity, but a process is an active entity.
- To accomplish its task ,a process needs certain resources like CPU time, memory, files and I/O devices.
- These resources are allocated to process either at the time of creation or when it is executing.

5) Peripheral or I/O device Management:

- Keep track of resources (device, channels, control units) attached to the system.
- Communication between these devices and CPU is observed by operating system.
- An operating system will have device drivers to facilitate I/O functions involving device like keyboard, mouse, monitor, disk, FDD, CD-ROM, printer etc.

6) *File Management:*

- A file is a collection of related information or record defined by the user.
- The operating system is responsible for various activities of file management are
 - i. Creation and deletion of files
 - ii. Creation and deletion of directories
 - iii. Manipulation of files and directories
 - iv. Mapping files onto secondary storage

7) *Secondary storage Management:*

- It is a larger memory used to store huge amount of data. Its capacity is much larger than primary memory. E.g. floppy disk, hard disk etc.
- The operating system is responsible for handling all the devices that can be done by these secondary storage management.
- The various activities are:
 - i. Free space management
 - ii. Storage allocation (allocation of storage space when new files have to be written).
 - iii. Disk scheduling (scheduling the request for memory access)

8) *Protection/Security Management:*

- If a computer system has a multiple processor, then the various processes must be protected of one another's activities.
- Protection refers to mechanism for controlling user access of programs or processes or user to resources defined by the computer system.

9) *Error detection and Recovery:*

- Error may occur during execution like divide by zero by a process, memory access violation, deadlock, I/O device error or a connection failure.
- The operating system should detect such errors and handles them.

3.0 PROCESS MANAGEMENT

Process

A process is basically an instance of a program in execution. The execution of a process must progress in a sequential fashion. A process is defined as an entity which represents the basic unit of work to be implemented in the system. To put it in simple terms, we write our computer programs in a text file and when we execute this program, it becomes a process which performs all the tasks mentioned in the program.

Note that:

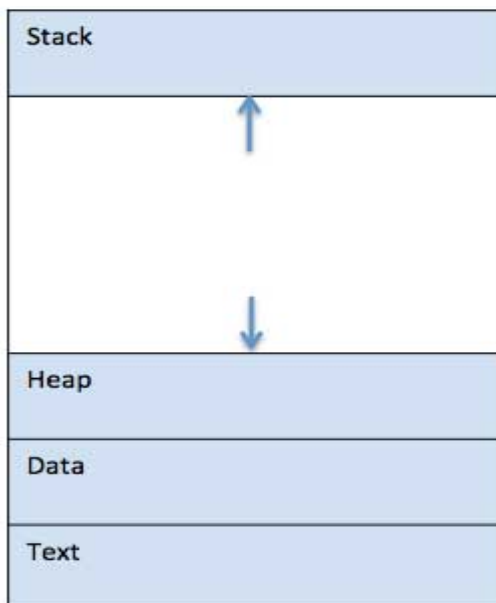
- Process is a program in execution
- Process is a currently executable task.
- Process execution must progress in a sequential manner.

<u>Process</u>	<u>Program</u>
i) A process is the set of executable instruction, those are the machine code.	i) It is a set of instruction written in programming language.
ii) Process is dynamic in nature.	ii) Program is static in nature.
iii) Process is an active entity.	iii) Program is a passive entity.
iv) A process resides in main memory.	iv) A program resides in secondary storage.
v) A process is expressed in assembly language or machine level language.	v) A program is expressed through a programmable language.
vi) The time period limited.	vi) Span time period is unlimited.

Thus, a computer program is a collection of instructions that performs a specific task when executed by a computer. When compared to a program, a process is thus a dynamic instance of a computer program.

A part of a computer program that performs a well-defined task is known as an **algorithm**. A collection of computer programs, libraries and related data are referred to as a **software**.

When a program is loaded into the memory and it becomes a process, it can be divided into four sections — stack, heap, text and data. The following image shows a simplified layout of a process inside main memory. That is, a process resides in memory through these four sections.



Stack

Stack section contains local variables. That is, it contains the temporary data such as method/function parameters, return address and local variables.

Heap

Heap section contains memory which will be dynamically allocated during runtime. That is dynamically allocated memory to a process during its run time.

Data

Data section contains global and static variable.

Text

Text section contains code or instruction. That is, it contains the current activity represented by the value of Program Counter and the contents of the processor's registers.

Program

A program is a piece of code which may be a single line or millions of lines. A computer program is usually written by a computer programmer in a programming language. E.g

```
#include <stdio.h>

int main() {
    printf("Hello, World! \n");
    return 0;
}
```

Process States

When a process executes, it passes through different states. These stages may differ in different operating systems, and the names of these states are also not standardized.

In general, a process can have one of the following five states at a time

1. New / Start: -

- ✓ This is the initial state when a process is first started/created.
- ✓ When the request is made by the user, the process is created.
- ✓ The newly created process moves into a new state.
- ✓ The process resides in secondary memory through a queue named as job queue or job pool.

2. Ready: -

- ✓ A process is said to be ready if it needs the processor to execute.
- ✓ The process is waiting to be assigned to a processor.
- ✓ Ready processes are waiting to have the processor allocated to them by the operating system so that they can run.
- ✓ Process may come into this state after New / Start state or while running, but interrupted by the scheduler to assign CPU to some other process.
- ✓ Out of total newly created processes, specified processes are selected and copied to temporary memory or main memory.
- ✓ In main memory they reside in a ready queue.

3. Running: -

- ✓ A process is said to be running if it currently has the CPU, i.e., actually using the CPU at that particular instant.
- ✓ Once the process has been assigned to a processor by the OS scheduler, the process state is set to running and the processor executes its instructions.

4. Waiting / Blocked: -

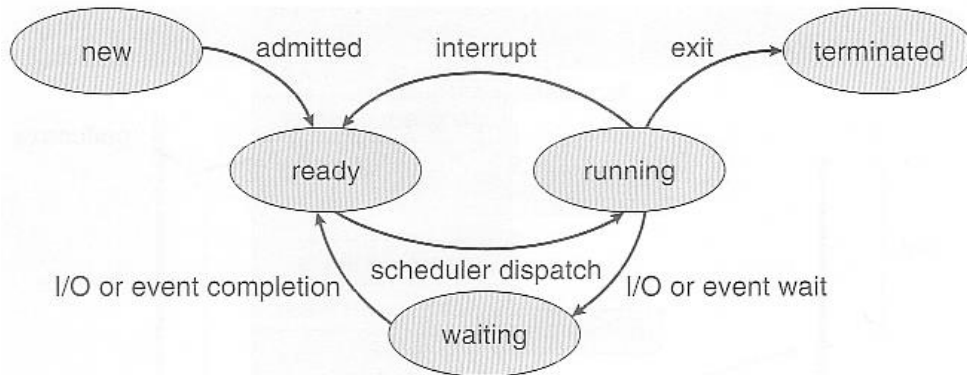
- ✓ Process moves into the waiting state if it needs to wait for a resource, such as waiting for user input, or waiting for a file to become available.
- ✓ When a process blocks, it does so because logically it cannot continue, typically because it is waiting for input that is not yet available.
- ✓ Formally, a process is said to be blocked if it is waiting for some event to happen (such as an I/O completion) before it can proceed.
- ✓ If a higher priority process arrives at the CPU during the execution of an ongoing process, then the CPU switches to the new process and current process enter into the waiting state.
- ✓ In this state a process is unable to run until some external event happens.

5. Terminated or Exit: -

- ✓ Once the process finishes its execution, or it is terminated by the operating system, it is moved to the terminated state where it waits to be removed from main memory.
- ✓ After completion of execution the process moves into the terminated state by exiting the system.
- ✓ The terminated state converts the process into a program.

- Sometimes operating system terminates the process due to the following reasons.

- ❖ Exceeding the time limit
- ❖ Input/output failure
- ❖ Unavailability of memory
- ❖ Protection error



Logically, the '**Running**' and '**Ready**' states are similar. In both cases the process is willing to run, only in the case of '**Ready**' state, there is temporarily no CPU available for it. The '**Blocked**' state is different from the '**Running**' and '**Ready**' states in that the process cannot run, even if the CPU is available.

Process Control Block (PCB)

To represent a process the operating System needs to group all the information of a process inside a data structure. This data structure is known as process control block (PCB).

Each process that runs in an operating system is assigned a process control block that holds information about the process, such as a unique process ID (a number used to identify the process), the saved state of the process, the process priority and where it is located in memory.

When a running process is interrupted by the processor after completing its allotted time, its state is saved in its process control block, its process state changed to ready and its priority adjusted.

The process priority is used to determine how often the process receives processor time. The operating system may run all processes with the same priority, or it may run some processes more often than others. Processes that have been waiting a long time for execution by the processor may have their priority increased so that they will be more likely to be executed in the future.

When a running process accesses an input or output device, or for some reason cannot continue, it is interrupted by the processor, the process state and associated data is saved in the associated process control block. The process state is changed to blocked and the priority adjusted.

When the scheduler decides the next task to run, it changes the process state of the selected process to running and loads the saved data associated with that process back into the processor.

Process Control Block (PCB) Defined.

A process control block or PCB is a data structure (a table) maintained by OS, that holds information about a process. Every process or program that runs needs a PCB. When a user requests to run a particular program, the operating system constructs a process control block for that program.

Typical information stored in a process control block includes the following;

- i. **Pointer**-It stores the starting address of the process.
- ii. **Process State**- This field stores or represent the current state of the process whether it s in ready/running/new/waiting/terminating.
- iii. **Process ID/Number**-Each process has uniqueID or serial no. Each process is shown an unique no. known as its Process ID or ProcessNumber.
- iv. **Program Counter**- It stores the address of the next instruction to be executed.
- v. **Register**-This field contains the name of the registers which are currently used by the processor.
- vi. **Scheduling Information**-This field stores the information about the scheduling algo.used by operating System for scheduling that process.
- vii. **Memory Management Information**-This field contains the value of thebase table, segment table and page table.
- viii. **Account Information**-This field contains the total no. processes, time slice period it used.
- ix. **File Management Information**- It stores various information about the files used by the process.
- x. **I/O Status Information**-It stores the information about various allocated I/O devices to the process, a list of open files & so on.

Pointer	Process state
Process number	
Program counter	
Registers	
Memory limits	
List of open files	
...	

Asymmetric Multiprocessing System (AMS)

This is a multiprocessing system in which each processor is assigned a specific task. For example, one processor is dedicated for handling user's requests, one processor is dedicated for running application program, and one processor is dedicated for running image processing and so on. In this system, one processor works as master processor, while other processors work as slave processors. The master processor controls the operations of system. It also schedules and distributes tasks among the slave processors. The slave processors perform the predefined tasks.

Symmetric Multiprocessing System (SMP)

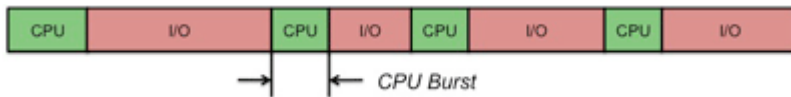
This is a multiprocessing system in which multiple processors work together on the same task. In this system, each processor can perform all types of tasks. All processors are treated equally and no master-slave relationship exists between the processors.

For example, different processors in the system can communicate with each other. Similarly, an I/O can be processed on any processor. However, I/O must be controlled to ensure that the data reaches the appropriate processor. Because all the processors share the same memory, so the input data given to the processors and their results must be separately controlled. Today all modern operating systems including Windows and Linux provide support for SMP.

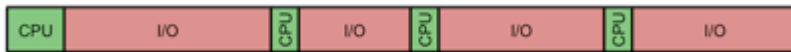
Process Scheduling

- ❖ The objective of multiprogramming is to improve the productivity of the computer by having process running at all times, to maximize CPU utilization.
- ❖ This is achieved by switching the CPU among processes so frequently that users can interact with each program while it is running while keeping the CPU busy at all the times.
- ❖ Process execution consists of a cycle of CPU execution (burst) and I/O wait.
- ❖ So, when two or more processes compete for the CPU at the same time, a choice has to be made.
- ❖ This procedure of determining the next process to be executed on the CPU is Process Scheduling.
- ❖ The module of the operating system that makes this decision is called the Scheduler.
- ❖ Scheduling is a fundamental operating system function, since almost all computer resources are scheduled before use.

If you look at any process, you'll notice that it spends some time executing instructions (computing) and then makes some I/O request, for example to read or write data to a file or to get input from a user. After that, it executes more instructions and then, again, waits on I/O. The period of computation between I/O requests is called the **CPU burst**.



Interactive processes spend more time performing and waiting for I/O and generally experience short CPU bursts: as indicated below.



Compute-intensive processes, conversely, spend more time running instructions and less time on I/O. They exhibit long CPU bursts: as indicated below



Most interactive processes, in fact, spend the vast bulk of their existence doing nothing but waiting on data. *As I write these notes on my laptop, I have over 30 processes running just under my user account. These includes a few browser windows, a word processor, spreadsheet, several shell windows, media player (since I'm listening to music while preparing these notes), and various monitors and utilities.* Most of the time, these processes collectively are using less than 3% of the CPU. This is not surprising since most of these programs are waiting for user input, a network message, or sleeping and waking up periodically to check some state.

Consider a quad-core 3.4 GHz Intel Core i7 processor. It can execute a peak rate of 13.6 billion instructions per second (four hyper threaded cores). It can run 136 billion instructions in the ten seconds it might take you to skim a web page before you click on a link — or 3.4 billion instructions in the quarter second it might take you to hit the next key as you're typing. The big idea in increasing overall throughput was the realization that we could switch the processor to running another process when a process has to do I/O. This is **multiprogramming**. The next big idea was realizing that you could preempt a process and

let another process run and do this quickly enough to give the illusion that many processes are running at the same time. This is **multitasking / time slicing**.

The assignment of physical processors to processes allows processors to accomplish work. The problem of determining when processors should be assigned and to which processes is called processor scheduling or CPU scheduling

When more than one process is runnable, the operating system must decide which one first. The part of the operating system concerned with this decision is called the scheduler, and the algorithm it uses is called the **scheduling algorithm**.

Process (CPU) Scheduling

Most systems have a large number of processes with short CPU bursts interspersed between I/O requests and a small number of processes with long CPU bursts. To provide good time-sharing performance, we may preempt a running process to let another one run. The **ready list**, also known as a **run queue**, in the operating system keeps a list of all processes that are ready to run and not blocked on some I/O or other system request, such as a semaphore (*a variable or abstract data type that provides a simple but useful abstraction for controlling access by multiple processes to a common resource in a parallel programming or multi user environment*). The entries in this list are pointers to the process control block, which stores all information and state about a process. When an I/O request for a process is complete, the process moves from the *waiting* state to the *ready* state and gets placed on the ready list.

Process scheduling consists of three sub functions:

- Scheduling Queue
- Process Scheduler
- Context Switching

Scheduling Queue

The operating system maintains several queues for efficient management of processes. There are four different types of scheduling queues, viz job, ready, device and blocked queues.

Job Queue

- ✓ When the process enters into the system, they are put into a job queue.
- ✓ This queue consists of all processes in the system on a secondary storage device e.g. hard disk.

Ready Queue

- ✓ From the job queue, the processes which are ready for execution are shifted to the main memory.
- ✓ In the main memory the processes are kept in the ready queue.
- ✓ In other words, the ready queue contains all those processes that are waiting for the CPU.

Device Queue

- ✓ Device queue is a queue for which a list of processes waiting for a particular I/O device.
- ✓ Each device has its own device queue.
- ✓ When a process requires some I/O operation, it is then taken out of the ready queue and kept under the device queue.

Blocked Queue

- ✓ This queue consists of all blocked or suspended processes in the system.

Process Scheduler

The **process scheduler** is the component of the operating system that is responsible for deciding whether the currently running process should continue running and, if not, which process should run next. There are four events that may occur where the scheduler needs to step in and make this decision:

1. The current process goes from the *running* to the *waiting* state because it issues an I/O request or some operating system request that cannot be satisfied immediately.
2. The current process terminates.
3. A timer interrupt causes the scheduler to run and decide that a process has run for its allotted interval of time and it is time to move it from the *running* to the *ready* state.
4. An I/O operation is complete for a process that requested it and the process now moves from the *waiting* to the *ready* state. The scheduler may then decide to move this *ready* process into the *running* state.

There are three types of Schedulers:

- ✓ Long Term Scheduler
- ✓ Short Term Scheduler
- ✓ Medium Term Scheduler

Long Term Scheduler

- ✓ Also called job scheduler; it works with the job queue.
- ✓ It selects processes from the job queue and loads them into the main memory for execution.
- ✓ The primary objective of the job scheduler is to control the degree of multiprogramming.
- ✓ When process changes the state from new to ready, then it is a long term scheduler.
- ✓ Long term scheduler selects a balanced mix of CPU bound and I/O bound processes.

Short Term Scheduler

- ✓ It is also called CPU scheduler or process scheduler.
- ✓ It selects the process from ready queue and allocates CPU to it.
- ✓ Main objective is to increase the system performance.
- ✓ This scheduler is frequently invoked as compared to long term scheduler.
- ✓ It is the change of ready state to running state of the process.

Medium Term Scheduler

- ✓ It is also known as Swapper.
- ✓ Sometimes the processes are removed from memory to reduce the degree of multiprogramming.
- ✓ After sometime the processes can be reintroduced into memory and execution can be continued where it was left off.
- ✓ This scheme is known as swapping.
- ✓ The medium term scheduler selects a process among the partially executed or unexecuted swapped out processes and swaps it back in the main memory.

Context Switching

A context switching is the mechanism to store and restore the state or context of a CPU in Process Control Block (PCB), so that a process execution can be resumed from the same point at a later time. Using this technique, a context switcher enables multiple processes to share a single CPU. Context switching is an essential part of a multitasking operating system feature.

```

graph LR
    CPU[CPU]
    P1_1[Process P1] --> S1[Saves P1 State]
    S1 -.-> R1[Restore P2 State]
    R1 --> P2_1[Process P2]
    P2_1 --> S2[Save P2 State]
    S2 -.-> R2[Restore P1 State]
    R2 --> P1_2[Process P1]
    S1 -.-> J1(( ))
    S2 -.-> J1
    R1 -.-> J2(( ))
    R2 -.-> J2
    J1 -.-> J2
  
```

- Program Counter
- Scheduling information
- Base and limit register value
- Currently used register
- Changed State
- I/O State information
- Accounting information

- ✓ Transferring the control of the CPU from one process to other requires saving the context of currently running process and loading the context of another ready process.
- ✓ This mechanism of saving and restoring the context is known as context switch.
- ✓ The portion of the PCB including the process state, memory management information and CPU scheduling information together constitutes the Context or State of the process.
- ✓ The switching periods depends upon the memory speed and the number of registers used.

A scheduler is a **preemptive** scheduler if it has the ability to get invoked by an interrupt and move a process out of a *running* state and let another process run. If a scheduler cannot take the CPU away from a process, then it is a **cooperative** or **non-preemptive** scheduler. Old operating systems such as Windows 3.1 or MacOS before OS X are examples of such schedulers

In this case once the CPU has been allocated to a process, the process keeps the CPU until it releases the CPU by terminating or by switching to the waiting state. That is when the process is completed or requires an I/O operation.

In this case CPU can be forcefully taken away. Under this scheduling, the CPU can be forcefully taken away by an interrupt from a higher priority process, completion of time slice, among other criteria.

The decisions that the scheduler makes concerning the sequence and length of time that processes may run is called the **scheduling algorithm** (or **scheduling policy**). These decisions are not easy ones, as the scheduler has only a limited amount of information about the processes that are ready to run. A good scheduling algorithm should:

- Be fair – give each process a fair share of the CPU, allow each process to run in a reasonable amount of time.
- Be efficient – keep the CPU busy all the time.
- Maximize throughput – service the largest possible number of jobs in a given amount of time; minimize the amount of time users must wait for their results.
- Minimize response time – interactive users should see good performance.
- Be predictable – a given job should take about the same amount of time to run when run multiple times. This keeps users *sane*.
- Minimize overhead – don't waste too many resources. Keep scheduling time and context switch time at a minimum.
- Avoid indefinite postponement – every process should get a chance to run eventually.
- Enforce priorities – if the scheduler allows a process to be assigned a priority, it should be meaningful and enforced.

It is clear that some of these goals are contradictory. For example, minimizing overhead means that jobs should run longer, thus hurting interactive performance. Enforcing priorities means that high-priority processes will always be favored over low-priority ones, causing indefinite postponement (“*hanging*”). These factors make scheduling a task for which there can be no perfect algorithm.

To make matters even more complex, the scheduler does not know much about the behavior of each process. As indicated earlier, some processes perform a lot of input/output operations but use little CPU time (e.g. web browsers, etc.). They spend much of their time in the blocked state in between little bursts of computation. The overall performance of these **I/O bound processes** is constrained by the speed of the I/O devices. **CPU-bound processes** spend most of their time computing (e.g. circuit simulators). Their execution time is determined by the speed of the CPU and the amount of CPU time they can get.

Preemptive scheduling allows the scheduler to control response times by taking the CPU away from a process that it decided has been running too long. It has more overhead than non-preemptive scheduling since it has to deal with the overhead of context switching (*computing process of storing and restoring the state (**context**) of a CPU so that execution can be resumed from the same point at a later time*) processes instead of allowing a process to run to completion or until the next I/O operation or other system call. However, it allows for higher degrees of concurrency and better interactive performance.

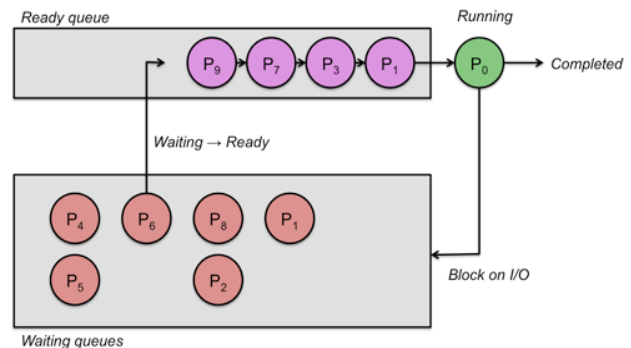
Scheduling Algorithms

The scheduling algorithm has the task of figuring out whether a process should be switched out for another process and which process should get to run next. The **dispatcher** is the component of the scheduler that handles the mechanism of actually getting that process to run on the processor. This requires loading the saved context of the selected process, which is stored in the process control block and comprises the set of registers, stack pointer, flags (status word), and a pointer to the memory mapping (typically a pointer to the page table). Once this context is loaded, the dispatcher switches to user mode via a *return from interrupt* operation that causes the process to execute from the location that was saved on the stack at the time that the program stopped running — either via an interrupt or a system call.

First-Come, First-Served Scheduling

Possibly the most straightforward approach to scheduling processes is to maintain a FIFO (first-in, first-out) ready queue. New processes go to the end of the queue. When the scheduler needs to run a process, it picks the process that is at the head of the queue. The scheduler is non-preemptive. If the process has to block on I/O, it enters the *waiting* state and the scheduler picks the process from the head of the queue. When I/O is complete and a process is ready to run again, it gets put at the end of the queue.

With first-come, first-served scheduling, a process with a long CPU burst will hold up other processes. Moreover, it can hurt overall throughput since I/O on processes in the *waiting* state may complete while the CPU bound process is still running. Now devices are not being used effectively. For increasing throughput, it would have been great if the scheduler instead could have briefly run some I/O bound process that could request some I/O and wait. Because CPU bound processes don't get preempted, they hurt interactive performance because the interactive process won't get scheduled until the CPU bound one has completed.



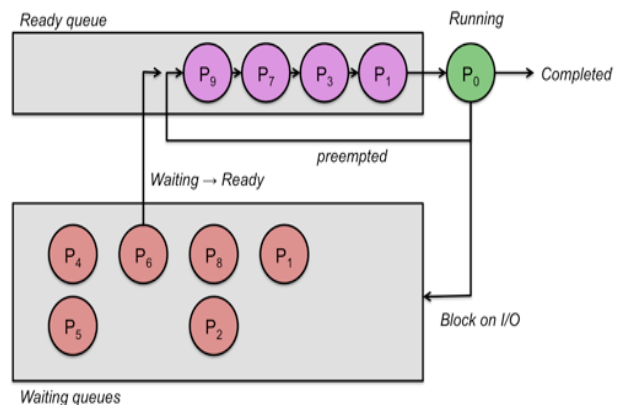
Advantage: FIFO scheduling is simple to implement. It is also intuitively fair.

Disadvantage: First-come, first-served scheduling is not preemptive. Because of this, it is not suitable for interactive jobs. Another drawback is that a long-running process will delay all jobs behind it.

Round robin scheduling

Round robin scheduling is a preemptive version of first-come, first-served scheduling. Processes are dispatched in a first-in-first-out sequence but each process is allowed to run for only a limited amount of time. This time interval is known as a **time-slice** or **quantum**. If a process does not complete or get blocked because of an I/O operation within the time slice, the time slice expires and the process is preempted, process gets blocked because of an I/O operation), it is then preempted. This preempted process is placed at the back of the ready list where it must wait for the processes that were already on the list to cycle through the CPU.

With round robin scheduling, interactive performance depends on the length of the quantum and the number of processes in the ready list (run queue). A very long quantum makes the algorithm behave very much like first come, first served scheduling since it's very likely that a process will block or complete before the time slice is up. A small quantum lets the system cycle through processes quickly. This is wonderful for interactive processes. Unfortunately, there is an overhead to context switching and having to do so frequently increases the percentage of system time that is used on context switching rather than real work.



Advantage: Round robin scheduling is fair in that every process gets an equal share of the CPU. It is easy to implement and, if we know the number of processes on the ready list, we can know the worst-case response time for a process.

Disadvantage: Giving every process an equal share of the CPU is not always a good idea. For instance, highly interactive processes will get scheduled no more frequently than CPU-bound processes.

4.0 PROCESS COORDINATION & SYNCHRONIZATION

Process synchronization refers to the idea that multiple processes are to join up or *handshake* at a certain point, in order to reach an agreement or commit to a certain sequence of action

Process Synchronization is the coordination of execution of multiple processes in a multi-process system to ensure that they access shared resources in a controlled and predictable manner. It aims to resolve the problem of race conditions and other synchronization issues in a concurrent system.

The main objective of process synchronization is to ensure that multiple processes access shared resources without interfering with each other and to prevent the possibility of inconsistent data due to concurrent access. To achieve this, various synchronization techniques such as semaphores, monitors, and critical sections are used.

In a multi-process system, synchronization is necessary to ensure data consistency and integrity, and to avoid the risk of deadlocks and other synchronization problems. Process synchronization is an important aspect of modern operating systems, and it plays a crucial role in ensuring the correct and efficient functioning of multi-process systems.

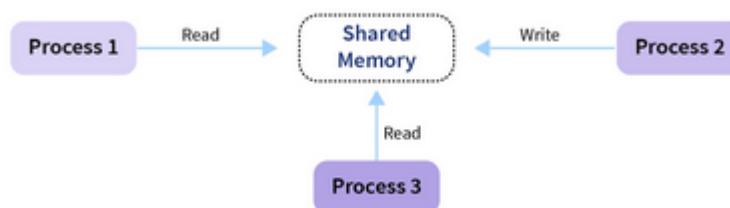
On the basis of synchronization, processes are categorized as one of the following two types:

- **Independent Process:** The execution of one process does not affect the execution of other processes.
- **Cooperative Process:** A process that can affect or be affected by other processes executing in the system.

Process synchronization problem arises in the case of Cooperative processes because resources are shared in Cooperative processes.

What is Race Condition?

When more than one process is executing the same code or accessing the same memory or any shared variable in that condition, there is a possibility that the output or the value of the shared variable will be wrong. E.g. When two students are paying fees to the same account, from different locations, the bank balance may be “*wrong*”. These two processes are racing to say that my output is correct. This condition is known as a **race condition**. Several processes access and process the same data concurrently, and then the outcome depends on the particular order in which the access takes place. A race condition is a situation that may occur inside a critical section. This happens when the result of multiple thread execution in the critical section differs according to the order in which the threads execute. Race conditions in critical sections can be avoided by use of proper synchronization.



Kamau and Onyango want to pay fees of Ksh 8,000 and Ksh 20,000 respectively to JKUAT KCB from different locations, apparently at the same time (concurrently). The account currently has Ksh 500,000 as the balance.

1. Kamau’s request (process) checks the balance: **500,000**

2. Onyango's request (process) checks the balance: **500,000**
3. Kamau's process credits the balance: **508,000**
4. Onyango's process credits the balance: **520,000**

This is a race condition because the final balance depends on the timing of the two operations. In a correct implementation, one of the operations should have been denied.

Effects of Race Condition in OS:

In an operating system, a race condition can have several effects, such as:

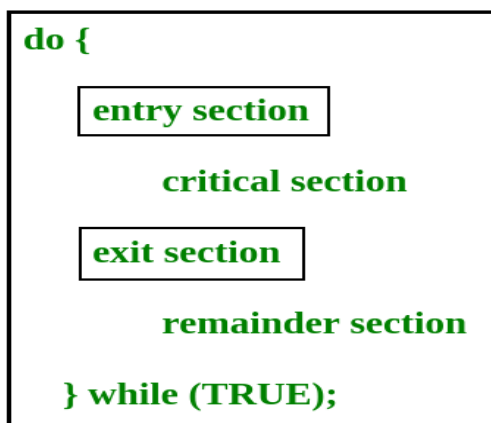
1. **Deadlocks:** - A race condition can cause deadlocks, where two or more processes are awaiting the completion of a job by one another but are unable to move forward. This can happen if two processes try to access the same resource simultaneously, and the operating system doesn't have a mechanism to ensure that only one can access the resource at a time.
2. **Data corruption:** - A race condition can cause data corruption, where two or more processes simultaneously try to write/update the exact memory location. This can cause the data to be overwritten or mixed up, resulting in incorrect results or program crashes.
3. **Security vulnerabilities:** - A race condition can also create security vulnerabilities in an operating system. For example, an attacker may be able to exploit a race condition to gain unauthorized access to a system or to escalate their privileges.
4. **Performance degradation:** - In some cases, a race condition can also cause performance degradation in an operating system. This can happen if multiple processes are competing for the same resources, such as CPU time or memory, and the operating system is unable to allocate these resources efficiently

Critical Section Problem

A critical section is a code segment that can be accessed by only one process at a time e.g. bank balance. The critical section contains shared variables that need to be synchronized to maintain the consistency of data variables. So, the critical section problem means designing a way for cooperative processes to access shared resources without creating data inconsistencies.

The critical section problem is to design a protocol that the processes can use to cooperate. Each process must request permission to enter its critical section. The section of code implementing this request is the entry section. The critical section may be followed by an exit section. The remaining code is the remainder section.

In the entry section, the process requests for entry in the Critical Section as illustrated below
Any solution to the critical section problem must satisfy three requirements



1. **Mutual Exclusion:** If a process is executing in its critical section, then no other process is allowed to execute in the critical section.
2. **Progress:** If no process is executing in the critical section and other processes are waiting outside the critical section, then only those processes that are not executing in their remainder section can participate in deciding which will enter the critical section next, and the selection cannot be postponed indefinitely.
3. **Bounded Waiting:** A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.

In Summary

Processes in a computer system execute programs to manipulate data. One process may or may not interact with another process to accomplish its task. Such interactions take place only through shared data. If two processes use two distinct sets of data items, then they are not considered interacting processes and their activities do not affect each other. Their behaviors are independent and do not influence each other. If a process A modifies a data item that another process B reads, then the behavior of B may depend on activities of A.

Here we say the two processes are interacting with one another through shared data. The behaviour of these interacting processes then depends on two factors: the relative speeds of the processes (what they do with shared data) and how they access the shared data.

The theme of process synchronization is the coordination of interacting processes, i.e, the orderly execution of their accesses to shared data.

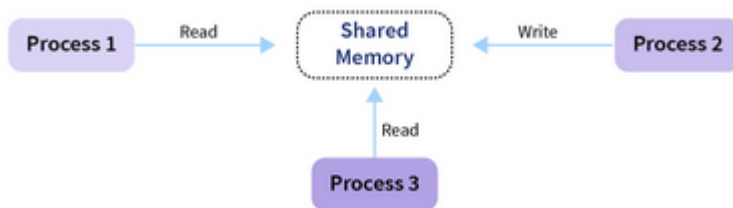
Note

1. The operating system or any application execution may be considered to be a collection of processes in which some interact and some others do not.
2. Synchronization is the single most important topic in highly concurrent systems such as operating systems, databases, and networks.

Process Synchronization

Synchronization is **the way by which processes that share the same memory space are managed in an operating system.**

It helps maintain the consistency of data by using variables or hardware so that only one process can make changes to the shared memory at a time



Modern operating systems are multi-process systems. They run many processes concurrently. Concurrency among processes is exploited to achieve better utilization of system resources. Two processes are said to be concurrent if their program executions overlap in real time. That is, the first operation execution of each process starts before the last operation execution of the other process is complete.

Process synchronization deals with various mechanisms to ensure orderly execution of cooperating processes that share a logical address space. The basic purpose of process synchronization is to maintain data consistency. A cooperating process is one that can affect or be affected by other processes executing in the system.

Concurrently running processes without any sort of data protection scheme can manipulate data and produce an incorrect result. Moreover, if the outcome of an execution depends on the particular order in which the data access takes place, a race condition can occur. So we need to implement some sort of synchronization of the processes.

Semaphores

A semaphore is a variable used to control access to a common resource by multiple threads and avoid critical section problems in a concurrent system such as a multitasking operating system.

A useful way to think of a semaphore as used in a real-world system is as a record of how many units of a particular resource are available, coupled with operations to adjust that record safely (i.e., to avoid race conditions) as units are acquired or become free, and, if necessary, wait until a unit of the resource becomes available.

Library Analogy

Suppose a physical library has ten identical private study rooms, to be used by one student at a time. Students must request a room from the front desk. If no rooms are free, students wait at the desk until someone relinquishes a room. When a student has finished using a room, the student must pass through the desk and indicate that the room is free.

This means, the clerk knows only the number of free rooms available. Students should use their room only while they have signed up for it and return it when they are done. When a student requests a room, the clerk decreases this number. When a student releases a room, the clerk increases this number. The room can be used for as long as desired, and so it is not possible to book rooms ahead of time.

In this scenario, the front desk count-holder represents a counting semaphore, the rooms are the resource, and the students represent processes/threads. The value of the semaphore in this scenario is initially 10, when all rooms are empty. When a student requests a room, they are granted access, and the value of the semaphore is changed to 9. After the next student comes, it drops to 8, then 7, and so on. If someone requests a room and the current value of the semaphore is 0, they are forced to wait until a room is freed (when the count is increased from 0). If one of the rooms was released, but there are several students waiting, then any algorithm can be used to select the one who will occupy the room (like FIFO or randomly picking one). And of course, a student must inform the clerk about releasing their room only after really leaving it

Mutual Exclusion in Concurrency

Mutual Exclusion is a property of process synchronization that states that *“no two processes can exist in the critical section at any given point of time”*

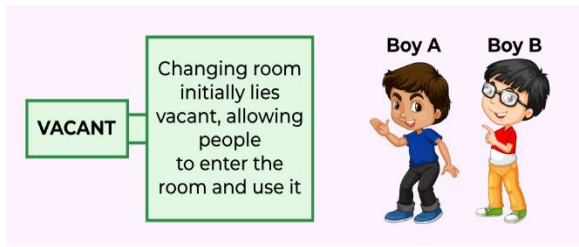
Mutual Exclusion refers to the property of a system where only one process or thread can access a shared resource at a time. This is crucial for ensuring data integrity and preventing race conditions, where multiple processes attempt to access and modify the same resource simultaneously, leading to unpredictable and potentially erroneous behavior.

In a concurrent system, multiple processes or threads may be running simultaneously and attempting to access shared resources such as variables, data structures, or hardware devices. Without proper mutual exclusion mechanisms in place, these processes can interfere with each other, leading to data corruption and inconsistencies.

There are various techniques for achieving mutual exclusion, such as using locks, semaphores, and atomic operations. These mechanisms ensure that only one process can access a shared resource at a time, while other processes are blocked or forced to wait until the resource becomes available.

Mutual exclusion is essential for maintaining the correctness and consistency of concurrent programs, particularly in multi-threaded and distributed systems. It plays a critical role in preventing race conditions and ensuring that concurrent processes can safely and effectively access and modify shared resources without conflicting with each other.

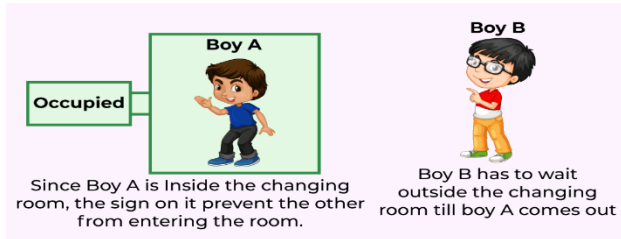
Example: In the clothes section of a supermarket, two people are shopping for clothes.



Boy, A decides upon some clothes to buy and heads to the changing room to try them out. Now, while boy A is inside the changing room, there is an 'occupied' sign on it – indicating that no one else can come in. Boy B has to use the changing room too, so he has to wait till boy A is done using the changing room.

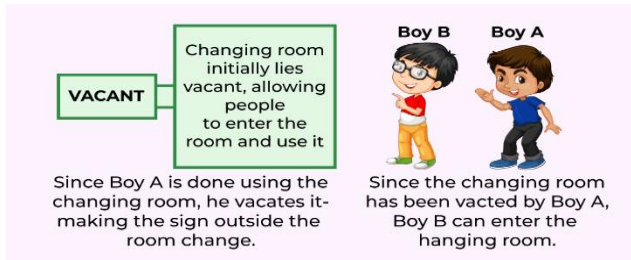
Once boy A comes out of the changing room, the sign on it changes from 'occupied' to 'vacant' – indicating that another person can use it. Hence, boy B proceeds to use the changing room, while the sign displays 'occupied' again.

The changing room is nothing but the critical section, boy A and boy B are two different processes, while the sign outside the changing room indicates the process synchronization mechanism being used.



Conclusion

In conclusion, mutual exclusion is a key concept in synchronization that ensures only one process accesses a shared resource at a time. This prevents conflicts and data corruption, making sure that processes run smoothly and correctly. By using mutual exclusion mechanisms, we can create stable and reliable systems that handle multiple processes efficiently.



So, mutual exclusion in concurrency is a fundamental principle that ensures only one process can access a shared resource at a time, preventing data corruption and race conditions in concurrent programs. It is a crucial concept for maintaining the integrity and reliability of concurrent systems and plays a vital role in the design and implementation of concurrent algorithms and data structures

5.0 MEMORY MANAGEMENT.

Memory Management is the process of controlling and coordinating computer memory, assigning portions known as blocks to various running programs to optimize the overall performance of the system.

It is the most important function of an operating system that manages primary memory. It helps processes to move back and forth between the main memory and secondary memory. It helps OS to keep track of every memory location, irrespective of whether it is allocated to some process or it remains free.

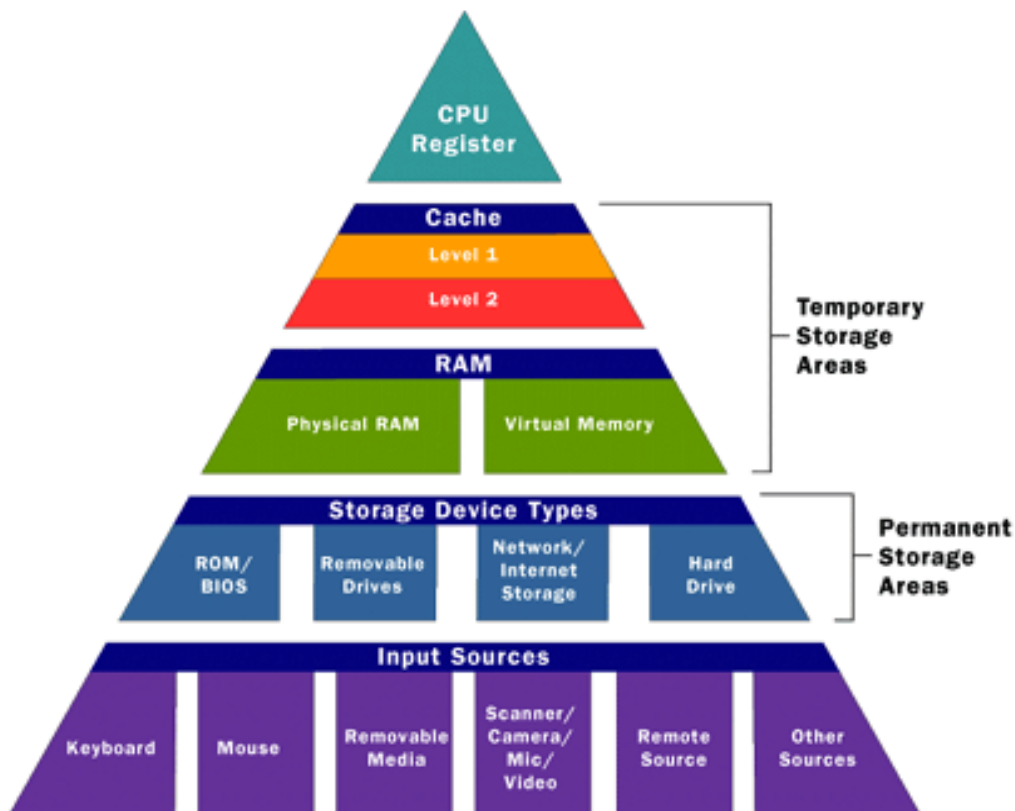
Fundamental Issues in Design and its Characteristics

In the Computer System Design, Memory Hierarchy is an enhancement to organize the memory such that it can minimize the access time. The Memory Hierarchy was developed based on a program behavior known as locality of references. The figure below shows the different levels of memory hierarchy design: This Memory Hierarchy Design is divided into 2 main types:

1. **External Memory or Secondary Memory –**
Comprising of Magnetic Disk, Optical Disk, Magnetic Tape i.e. peripheral storage devices which are accessible by the processor via I/O Module.
2. **Internal Memory or Primary Memory –**
Comprises of Main Memory, Cache Memory & CPU registers. This is directly accessible by CPU.

The following characteristics of Memory Hierarchy Design can be inferred from the figure:

1. **Capacity:**
It is the global volume of information the memory can store. As we move from top to bottom in the Hierarchy, the capacity increases.
2. **Access Time:**
It is the time interval between the read/write request and the availability of the data. As we move from top to bottom in the Hierarchy, the access time increases.
3. **Performance:**
Earlier when the computer system was designed without Memory Hierarchy design, the speed gap increases between the CPU registers and Main Memory due to large difference in access time. This results in lower performance of the system and thus, enhancement was required. This enhancement was made in the form of Memory Hierarchy Design because of which the performance of the system increases. One of the most significant ways to increase system performance is minimizing how far down the memory hierarchy one has to go to manipulate data.
4. **Cost per bit:**
As we move from bottom to top in the Hierarchy, the cost per bit increases i.e. Internal Memory is costlier than External Memory.



Memory

A computer generally has a mixture of memory types--such as main memory (RAM, ROM), cache, hard disks, and flash disks--each with a different size and access time. Programs often cannot be stored in their entirety in the main memory. Thus, they are stored in hard disks that have a large capacity but a slower access time, and then appropriate segments of the program are transferred for fast execution to the main memory. Small parts of these segments are further transferred into cache. For fast and reliable processing of a large number of programs, the operating system must constantly shuffle around the program segments and the intermediate processing results among these memories.

In some computers, users can access hard disks as an extension of the main memory without being aware of the distinction between the two levels of storage. Users can then freely employ far greater memory space than is available in the main memory. This is achieved by treating the memory addresses of the hard disks as a consecutive extension of the memory addresses of the main memory. This technique is called virtual memory and is managed by the operating system. Paging and segmentation are two common methods of realizing virtual memory. Blocks of either fixed size (pages) or variable size (segments) are moved as needed from hard disks to the main memory. As the blocks are moved in, their virtual addresses are mapped to real (physical) addresses in the main memory.

Therefore (in summary), a computer can address more memory than the amount physically installed on the system. This extra memory is actually called virtual memory and it is a section of a hard disk that's set up to emulate the computer's RAM. Paging technique plays an important role in implementing virtual memory.

Note that primary (main) memory provides storage that can be accessed directly by the CPU. That is, for a program to be executed (run), it must be in the main memory.

The major activities of an operating in regard to memory-management are:

- Keeps tracks of primary memory, i.e., what part of it are in use by whom, what part are not in use.
- In multiprogramming, the OS decides which process will get memory when and how much.
- Allocates the memory when a process requests it to do so.
- De-allocates the memory when a process no longer needs it or has been terminated.

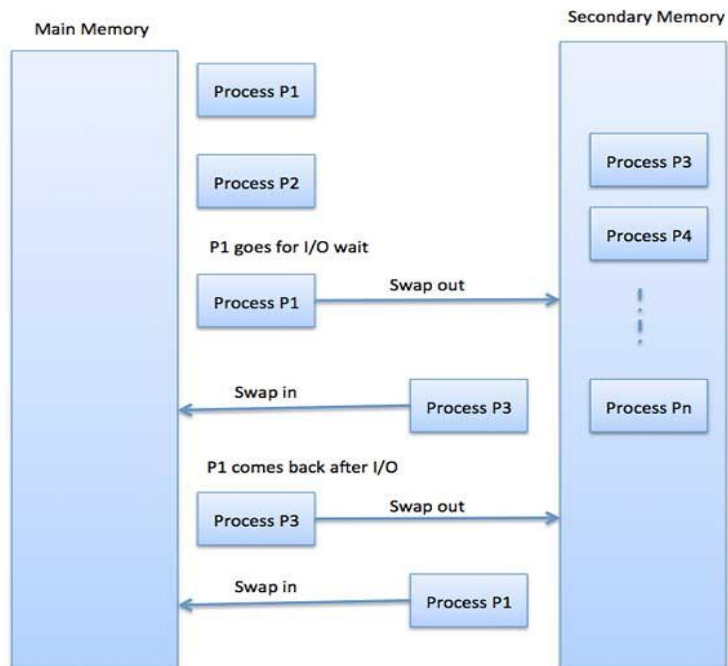
Memory management is the functionality of an operating system which handles or manages primary memory and moves processes back and forth between main memory and disk during execution. Memory management keeps track of each and every memory location, regardless of either it is allocated to some process or it is free. It checks how much memory is to be allocated to processes. It decides which process will get memory at what time. It tracks whenever some memory gets freed or unallocated and correspondingly it updates the status.

Swapping

Swapping is a mechanism in which a process can be swapped temporarily out of main memory (or move) to secondary storage (disk) and make that memory available to other processes. At some later time, the system swaps back the process from the secondary storage to main memory.

Though performance is usually affected by swapping process but it helps in running multiple and big processes in parallel and that's the reason **Swapping is also known as a technique for memory compaction.**

The total time taken by swapping process includes the time it takes to move the entire process to a secondary disk and then to copy the process back to memory, as well as the time the process takes to regain main memory.



Let us assume that the user process is of size 2048KB and on a standard hard disk where swapping will take place has a data transfer rate around 1 MB per second. The actual transfer of the 1000K process to or from memory will take

$$\begin{aligned}
 &2048\text{KB} / 1024\text{KB per second} \\
 &= 2 \text{ seconds} \\
 &= 2000 \text{ milliseconds}
 \end{aligned}$$

Now considering in and out time, it will take complete 4000 milliseconds plus other overhead where the process competes to regain main memory.

Fragmentation

As processes are loaded and removed from memory, the free memory space is broken into little pieces. It happens after sometimes that processes cannot be allocated to memory blocks considering their small size and memory blocks remains unused. This problem is known as Fragmentation.

Fragmentation is of two types –

External fragmentation

Total memory space is enough to satisfy a request or to reside a process in it, but it is not contiguous, so it cannot be used.

Internal fragmentation

Memory block assigned to process is bigger. Some portion of memory is left unused, as it cannot be used by another process.

The following diagram shows how fragmentation can cause waste of memory and a compaction technique can be used to create more free memory out of fragmented memory –

Fragmented memory before compaction



External fragmentation can be reduced by compaction or shuffle memory contents to place all free memory together in one large block. To make compaction feasible, relocation should be dynamic.

Memory after compaction



The internal fragmentation can be reduced by effectively assigning the smallest partition but large enough for the process.

Virtual Memory

Virtual Memory is a storage allocation scheme in which secondary memory can be addressed as though it were part of the main memory. It is a memory management technique used by operating systems to give the appearance of a large, continuous block of memory to applications, even if the physical memory (RAM) is limited. It allows the system to compensate for physical memory shortages, enabling larger applications to run on systems with less RAM.

It is a memory hierarchy, consisting of a computer system's memory and a disk, that enables a process to operate with only some portions of its address space in memory. A virtual memory is what its name indicates- it is an illusion of a memory that is larger than the real memory.

There are two main types of virtual memory: **Paging** and **Segmentation**.

Paging

Paging divides memory into small fixed-size blocks called pages. When the computer runs out of RAM, pages that aren't currently in use are moved to the hard drive, into an area called a swap file. The swap file acts as an extension of RAM. When a page is needed again, it is swapped back into RAM, a process known as page swapping. This ensures that the operating system (OS) and applications have enough memory to run.

The primary concept behind paging is to break each process into individual pages. Thus, the primary memory would also be separated into frames.

NB: Page = Processes/jobs divided into equal parts. (Block of consecutive virtual memory).

Frame = RAM divided into equal parts. (Block of consecutive physical memory).

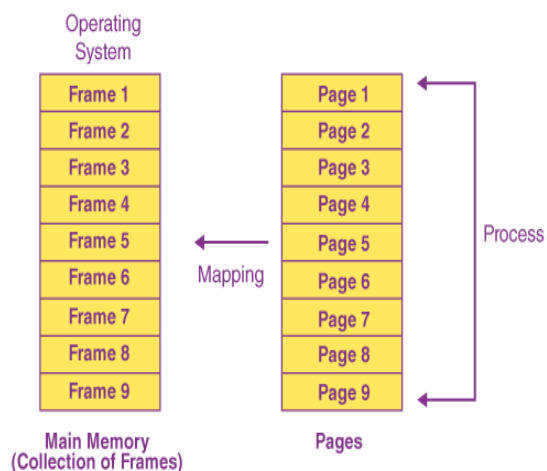
One page of the process must be saved in one of the given memory frames. These pages can be stored in various memory locations, but finding contiguous frames/holes is always the main goal. Process pages are usually only brought into the main memory when they are needed; else, they are stored in the secondary storage.

The frame sizes may vary depending on the OS. Each frame must be of the same size. Since the pages present in paging are mapped on to the frames, the page size should be similar to the frame size.

Example

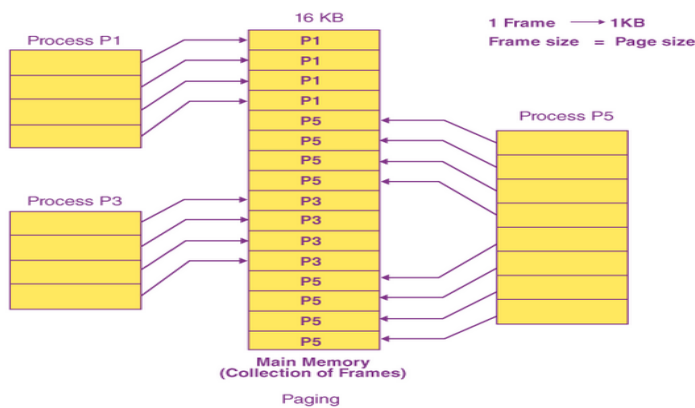
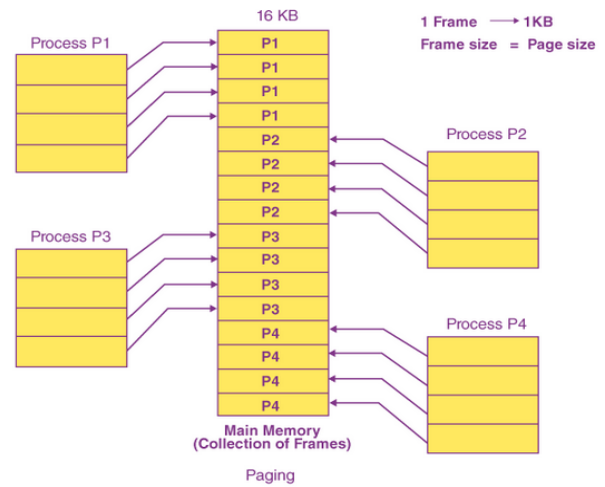
Assuming that the main memory is 16 KB and the frame size is 1 KB, the main memory will be partitioned into a collection of 16 1 KB frames. P1, P2, P3, and P4 are the four processes in the system, each of which is 4 KB in size. Each process is separated into 1 KB pages, allowing one page to be saved in a single frame.

Because all of the frames are initially empty, the pages of the processes will be stored in a continuous manner. The graphic on the right depicts frames, pages, and the mapping between them.



Consider the case when P2 and P4 are shifted to the waiting state after a period of time. Eight frames are now empty, allowing other pages to be loaded in their stead. Inside the ready queue is the process P5, which is 8 KB (8 pages) in size.

Given that we have 8 noncontiguous frames accessible in memory, paging allows us to store the process in many locations. As a result, we can load the process P5 instead of P2 and P4



Advantages and Disadvantages of Paging

- Paging reduces external fragmentation, but still suffer from internal fragmentation.
- Paging is simple to implement and assumed as an efficient memory management technique.
- Due to equal size of the pages and frames, swapping becomes very easy.
- Page table requires extra memory space, so may not be good for a system having small RAM.

Paging Policies

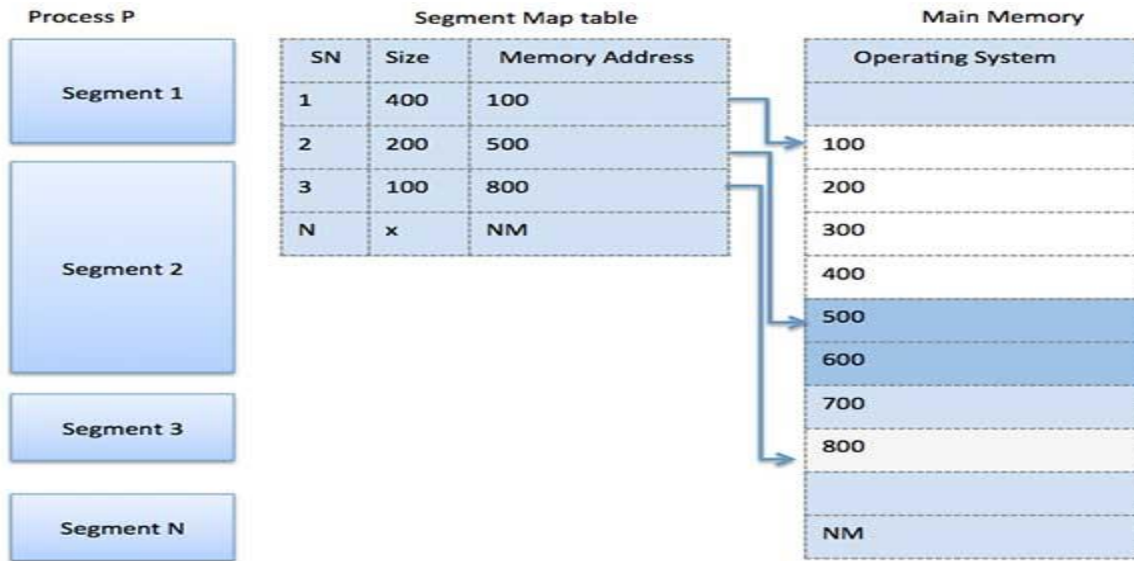
- **Fetch Policy:** It decides when a page should be loaded into memory.
- **Replacement Policy:** It decides which page in memory should be replaced.
- **Placement Policy:** It decides where in memory should a page be loaded.

Segmentation

Segmentation divides virtual memory into segments of different sizes. Segments that aren't currently needed can be moved to the hard drive. The system uses a segment table to keep track of each segment's status, including whether it's in memory, if it's been modified, and its physical address. Segments are mapped into a process's address space only when needed.

In segmentation, each job is divided into several segments of different sizes, one for each module that contains pieces that perform related functions. Each segment is actually a different logical address space of the program.

When a process is to be executed, its corresponding segmentation are loaded into non-contiguous memory though every segment is loaded into a contiguous block of available memory. Segmentation memory management works very similar to paging but here segments are of variable-length where as in paging pages are of fixed size.



Partitioning:

Partitioning divides primary memory into various memory partitions, which is mostly contiguous areas of memory. Every partition stores all the information for a specific task or job. This method consists of allotting a partition to a job when it starts & unallocated when it ends.

The total memory is divided into various partitions of the same size or different sizes. This helps to accommodate a number of programs in the memory. The partition can be fixed i.e. remains the same for all the programs in the memory or variable i.e. memory is allocated when a program is loaded onto the memory. The latter approach causes less wastage of memory but in due course of time, it may become fragmented.

Memory Management Algorithms

Memory management algorithms can vary widely among different hardware architectures and operating systems, depending on their goals and constraints. Examples include first-fit, best-fit, next-fit, demand paging, least recently used (LRU) algorithm, etc.

First-fit algorithm is a simple and fast partitioning algorithm that allocates the first available partition large enough to fit the request; however, it can cause internal fragmentation and poor memory utilization.

Best-fit algorithm is a partitioning algorithm that allocates the smallest available partition large enough to fit the request, reducing internal fragmentation but increasing external fragmentation and allocation time.

Next-fit algorithm is a partitioning algorithm that allocates the next available partition large enough to fit the request, reducing allocation time and external fragmentation but increasing internal fragmentation and memory wastage.

Demand paging algorithm is a paging algorithm that loads pages into main memory only when they are needed by processes or programs, improving memory utilization and response time but increasing memory access time and page fault rate.

Least recently used (LRU) algorithm is a paging algorithm that swaps out pages not used for longest time, optimizing memory access time and page fault rate but increasing memory management overhead.

6.0 FILE SYSTEM

File Management

The operating system manages the files, folders, and directory systems on a computer. Any data on a computer is stored in the form of files and the operating system keeps the information about all of them using File Allocation Table (FAT). The FAT stores general information about files like filename, type (text or binary), size, starting address, and access mode (sequential/indexed sequential/ direct/ relative). The file manager of the operating system helps to create, edit, copy, allocate memory to the files, and also updates the FAT. The operating system also takes care that files are opened with proper access rights to read or edit them.

The operating system is also responsible for keeping track of the files and directories that reside on the secondary (hard, flush, floppy disks, etc).

A file is a collection of related information defined by its creator. Computer can store files on the disk (secondary storage), which provide long term storage. Some examples of storage media are magnetic tape, magnetic disk and optical disk. Each of these media has its own properties like speed, capacity, data transfer rate and access methods.

A file system is normally organized into directories to ease their use. These directories may contain files and other directories.

The major activities of an operating system in regard to file management are

1. The creation and deletion of files.
2. The creation and deletion of directories.
3. The mapping of files onto secondary storage.
4. The backup of files on stable storage media.

A file is a named collection of related information that is recorded on secondary storage such as magnetic disks, magnetic tapes and optical disks. In general, a file is a sequence of bits, bytes, lines or records whose meaning is defined by the files creator and user.

File Structure

A File Structure should be according to a required format that the operating system can understand.

- A file has a certain defined structure according to its type.
- A text file is a sequence of characters organized into lines.
- A source file is a sequence of procedures and functions.
- An object file is a sequence of bytes organized into blocks that are understandable by the machine.
- When operating system defines different file structures, it also contains the code to support these file structure.

File Type

File type refers to the ability of the operating system to distinguish different types of file such as text files source files and binary files etc. Many operating systems support many types of files. Operating system like MS-DOS and UNIX have the following types of files –

Ordinary files

- These are the files that contain user information.
- These may have text, databases or executable program.
- The user can apply various operations on such files like add, modify, delete or even remove the entire file.

Directory files

- These files contain list of file names and other information related to these files.

File Access Mechanisms

File access mechanism refers to the manner in which the records of a file may be accessed. There are several ways to access files –

- Sequential access
- Direct/Random access
- Indexed sequential access

Sequential access

A sequential access is that in which the records are accessed in some sequence, i.e., the information in the file is processed in order, one record after the other. This access method is the most primitive one. Example: Compilers usually access files in this fashion.

Direct/Random access

- Random access file organization provides, accessing the records directly.
- Each record has its own address on the file with by the help of which it can be directly accessed for reading or writing.
- The records need not be in any sequence within the file and they need not be in adjacent locations on the storage medium.

Indexed sequential access

- This mechanism is built up on base of sequential access.
- An index is created for each file which contains pointers to various blocks.
- Index is searched sequentially and its pointer is used to access the file directly.

Space Allocation

Files are allocated disk spaces by operating system. Operating systems deploy following three main ways to allocate disk space to files.

- Contiguous Allocation
- Linked Allocation
- Indexed Allocation

Contiguous Allocation

- Each file occupies a contiguous address space on disk.
- Assigned disk address is in linear order.
- Easy to implement.
- External fragmentation is a major issue with this type of allocation technique.

Linked Allocation

- Each file carries a list of links to disk blocks.
- Directory contains link / pointer to first block of a file.
- No external fragmentation
- Effectively used in sequential access file.
- Inefficient in case of direct access file.

Indexed Allocation

- Provides solutions to problems of contiguous and linked allocation.
- An index block is created having all pointers to files.

- Each file has its own index block which stores the addresses of disk space occupied by the file.
- Directory contains the addresses of index blocks of files.

File System Manipulation

The output of a program may need to be written into new files or input taken from some files. The operating system provides this service. The user does not have to worry about secondary storage management. User gives a command for reading or writing to a file and sees his/her task accomplished. Thus operating system makes it easier for user programs to accomplish their task.

This service involves secondary storage management. The speed of I/O that depends on secondary storage management is critical to the speed of many programs and hence it is best relegated to the operating systems to manage it than giving individual users the control of it. It is not difficult for the user-level programs to provide these services but for the above-mentioned reasons it is best if this service is left with operating system.

For most users, the file system is the most visible aspect of an operating system, apart from the user interface. Files store programs and data.

The operating system implements the abstract 'file' concept by managing I/O devices, for example, hard disks. Files are usually organised into directories to make them easier to use. Files also usually have some form of protections.

The file system provides: (1) An interface to the file objects for both programmers and users and (2) The policies and mechanisms to store files and their attributes on permanent storage devices.

System Files

System files are files that the OS requires in order to function properly. Each system file has a function that alters, manages, or runs part of the OS or other system files. Contrast this with a data or *application file*, such as a saved game, document, or image file. The existence of an application file will not alter the behavior of the OS. You can change, create, or delete a non-system file without affecting the performance of the computer.

In most cases, a deleted, missing, or corrupt system file will result in an improperly functioning OS feature or an OS that will not function at all. A thorough knowledge of the OS system files and their functions will allow you to pinpoint and resolve OS problems based on the behavior of the computer or on the error messages you receive.

Many of the OS system files are location-dependent. That is, they will not be loaded and properly executed if they are not in a specific location. For this reason, and to keep them from being accidentally deleted, most system files are *hidden*. This means that they will not be displayed in the My Computer or Windows Explorer navigation windows.

File Control Blocks

File control blocks (FCB), sometimes referred to as file descriptors, are data structures that hold information about a file. When an operating system needs to access a file, it creates an associated file control block to manage the file.

The structure of the file control block differs between operating systems, but most file control blocks include the following parts

- Filename, Location of file on secondary storage, Length of file, Date and time of creation or last access

7.0 DEADLOCKS

Introduction

Some I/O media such as disks are easily sharable. However, some I/O media such as tape drives or printers cannot. So, they have to be allocated for one process exclusively.

Computer processing is full of resources that can only be used by one process at a time. Unpredictable results can occur if two or more processes use the same resource at the same time. The OS control this problem by arbitrating resource requests by granting exclusive access and managing access to system resources. By allowing processes to have exclusive access to resources, problems may occur: – *Deadlock*.

Whenever two or more processes try to access a resource that is in use by some other process then it is said to be a deadlock situation. A deadlock is a situation in which two or more competing actions are each waiting for the other to finish, and thus neither ever does. Deadlock causes all processes involved in the deadlock to become permanently blocked. Each process is waiting on resources owned by other processes that are also deadlocked. Deadlocks occur when processes have been granted exclusive access to resources.

A set of process is in a deadlock state if each process in the set is waiting for an event that can be caused by only another process in the set (including itself). In other words, each member of the set of deadlock processes is waiting for a resource that can be released only by a deadlock process. None of the processes can run, none of them can release any resources, and none of them can be awakened. It is important to note that the number of processes and the number and kind of resources possessed and requested are unimportant.

The resources may be either physical or logical. Examples of physical resources are Printers, Tape Drives, Memory Space, and *CPU* Cycles. Examples of logical resources are Files, processes, etc.

Deadlock defined

In a multiprogramming environment, several processes may compete for a finite number of resources. When a process requests resources and if the resources are not available at that time, the process enters into a wait state. Waiting processes may never again change their state, because the resources they have requested are held by other waiting processes. This situation is known as deadlock.

Assume two processes p1 and p2 running simultaneously. Half way through, p1 requests the operating system for a file in the tape drive, and p2 requests the operating system for the printer. Assume also that both requests are granted. After a while, p1 requests for the printer without giving the tape drive. Simultaneously p2 requests the same tape drive without giving up control of the printer. In this situation, both processes cannot proceed. P1 will wait until p2 releases the printer. But that can happen only if p2 can proceed further and finish off its processing with the printer. And this can happen only if p2 gets the same tape drive that p1 is holding. P2 can get the tape drive only if p1 can proceed further and completes its work with the tape drive. That cannot happen too, unless p1 gets the printer which p2 is holding. This situation is “deadlock”.